

Achieve What You Imagined: Learning to Align Actions with Visual Plans

Anonymous Author

Abstract—World-action models can jointly predict future visual observations and robot actions. However, discrepancies may exist between their visual predictions and the consequences implied by generated actions. We observe that WAMs can often generate visually plausible task-completion outcomes before producing action sequences that reliably achieve them. Consequently, we treat the WAM-generated visual prediction as a goal-conditioned visual proposal rather than a directly executable plan. We use a frozen action-conditioned world model to predict action-conditioned consequences and construct feedback based on consistency between the two future predictions and alignment with the terminal goal. Leveraging this feedback, we employ flow policy optimization (FPO) to optimize the action head of the WAM. This framework avoids online robot interaction and additional training of task-specific reward models. Across four real-world UR5 manipulation tasks, our method increases the mean success rate from 43.4% to 75.1%, compared with 61.4% for $\pi_{0.5}$. These results show that cross-model prediction discrepancy can provide useful feedback for improving robot policies under the evaluated manipulation tasks. Website: <https://imagine-to-achieve.github.io/>

I. INTRODUCTION

Robot learning is increasingly moving from task-specific controllers toward large pretrained policies that enable transfer across tasks and robot embodiments. Vision-language-action (VLA) models map observations and instructions directly to actions [1], [2], [3], while world-action models (WAMs) additionally predict future visual observations associated with generated actions [4], [5]. Reinforcement learning is a common approach for post-training such policies, but its effectiveness depends on obtaining informative rollout feedback, which remains challenging in low-data settings. Binary terminal rewards are well aligned with the task but often too sparse early on: when a weak base policy fails across sampled rollouts, they provide limited task-directed signal [6], [7]. Task-matched demonstrations raise the initial success rate but shift the burden back to collecting expert data [8]. Learned visual or vision-language evaluators [9], [10] provide denser feedback, but require additional pipelines for data preparation, training, calibration, and deployment [11]. We therefore ask whether a pretrained policy can obtain dense, task-relevant feedback from its predictive models, without training an additional task-specific reward model or requiring online robot interaction.

A WAM provides a source of task-level visual prediction, since its generated video can depict possible task-completion outcomes. During training, however, the two output streams can diverge: the generated video may show task completion, while the action chunk generated alongside

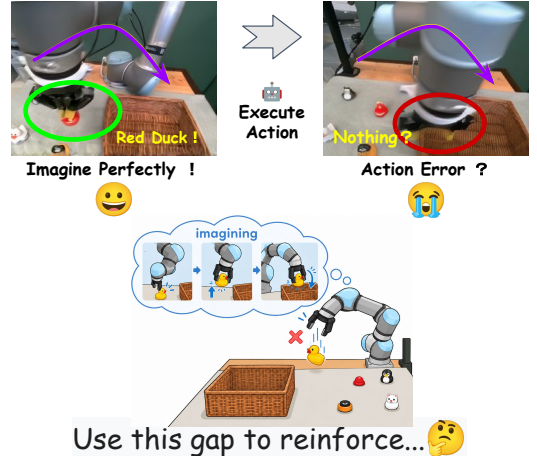


Fig. 1. **Imagination-consequence gap.** From the same instruction and initial state, the world-action model imagines a trajectory that completes the task, but its generated actions fail to execute it. The model already knows what success looks like before it can act on it — we turn this gap into a training signal.

it may not lead to the corresponding predicted consequence (Fig. 1). The model can develop stronger task-completion visual prediction before generating action sequences that reliably lead to such outcomes, suggesting an opportunity to improve action generation. However, the world model’s visual predictions (videos) do not embed physics information or provide feasibility validation, indicating that a visual plan is not necessarily a valid motor plan. Under these constraints, we treat the WAM-generated visual prediction as a goal-conditioned visual proposal and use a separately pretrained, frozen action-conditioned world model to estimate the consequences of generated actions. The feedback signal measures trajectory-level agreement between the two predictions and alignment of the final predicted frame with the task goal. Because the video head and consequence model are frozen during optimization, the policy cannot directly modify the predictive models that generate the feedback signal. This provides dense rollout-level feedback without online robot interaction, reward labels, or a separately trained success detector, which can then be used for policy optimization.

That step is not straightforward for a flow-based WAM. Existing RL methods for flow policies often rely on explicit sampling formulations or stochastic reformulations to obtain tractable optimization objectives [12], [13]. Cosmos3 instead samples video and actions using UniPC [5], a high-order multi-step solver for which such per-step likelihood formulations are not directly available. We therefore adapt Flow Policy Optimization [14], which builds an advantage-weighted surrogate from conditional flow-matching losses

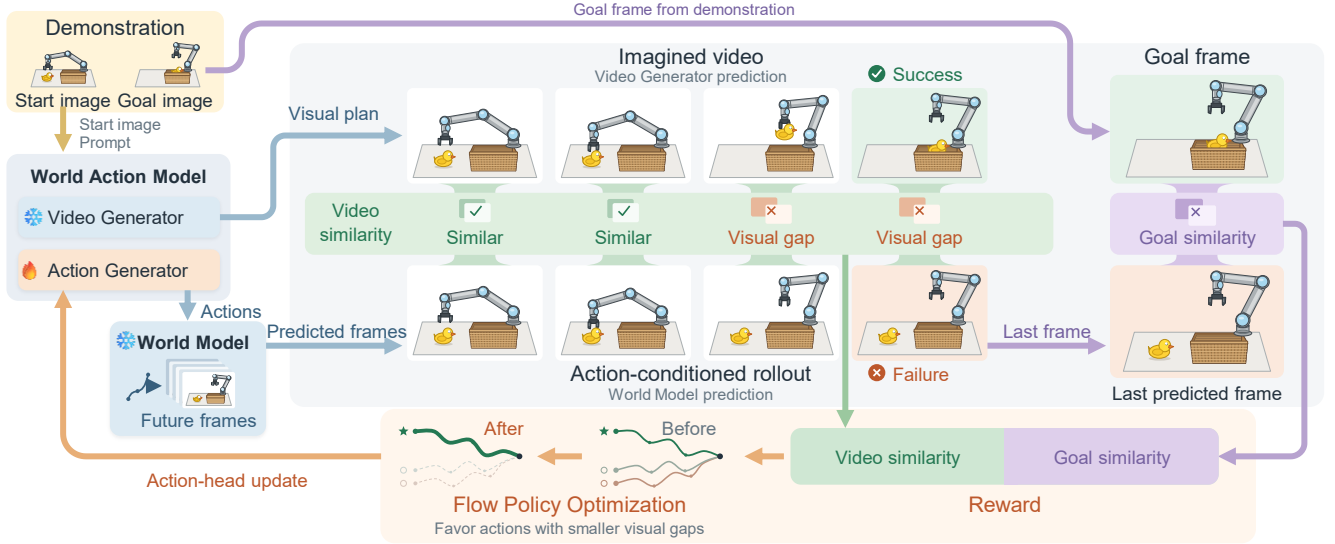


Fig. 2. **Overview of imagination-action alignment for robot manipulation.** From the start image and instruction, WAM rolls out future video and actions. A frozen action-conditioned world model predicts what those actions would actually produce and supplies the next observation. We evaluate action consequences using trajectory similarity between the two videos and goal similarity against a demonstration goal frame. Because both video predictors stay frozen (snowflake), the policy can only raise these rewards by changing its actions. These rewards benefit FPO, which updates the action head (flame) using a clipped surrogate objective.

while treating the sampler as a black box. We adapt Flow Policy Optimization to optimize a flow-based robot policy under a pretrained high-order multi-step sampler, enabling optimization through Cosmos3’s native UniPC sampling procedure while updating only the action head.

The main contributions of this work are:

- We introduce a world-model-guided RL framework that exploits the discrepancy between WAM-generated visual predictions and action-conditioned consequences predicted by a frozen world model to refine robot policies without online robot interaction.
- We identify an imagination–consequence gap in world-action models and leverage it as dense rollout-level feedback. A frozen pretrained action-conditioned world model provides feedback while avoiding direct updates to the feedback-generating models.
- We adapt Flow Policy Optimization for flow-based world-action policies with pretrained high-order multi-step samplers, enabling action-head-only optimization under the native UniPC sampling procedure.

II. RELATED WORK

Video and world action models for control: Video and world models support robot control by predicting future observations, states, or task-relevant representations for planning and manipulation. One line generates future videos as goal-conditioned visual predictions and maps them into actions with a separate inverse dynamics model [4], [15], [16], decoupling high-level prediction from low-level control but introducing the challenge of maintaining consistency between predicted visual trajectories and generated actions. A second line predicts observations and actions jointly in a unified model, sharing representations or parameters across both modalities [17], [18], [19], [20]. A third line predicts compact latent world states [21], [22], [23], [24], improving

efficiency while providing less direct access to pixel-level observations for visual consistency evaluation. We follow the joint world-action modeling paradigm and build on Cosmos3 [5]. However, existing joint models optimize visual and action generation objectives separately, while explicit alignment between predicted visual outcomes and action-conditioned consequences remains insufficiently studied.

World models for action outcome prediction: Action-conditioned world models predict future observations conditioned on states and actions, providing estimates of possible action consequences. Existing approaches either use explicit action conditioning with control semantics [25], [26], [27], [28] or learn latent action representations from video [29], [30], [31]. While latent-action models enable action-free learning and cross-embodiment transfer, their latent interfaces are not directly aligned with explicit robot commands. Ctrl-World follows the explicit-action paradigm and supports long-horizon prediction through multi-view prediction, action conditioning, and pose-based memory retrieval [25]. We use a frozen pretrained action-conditioned world model as an external consequence predictor, whose rollouts provide alignment feedback between action consequences and WAM visual predictions.

RL for generative robot policies: Reinforcement learning for generative robot policies optimizes pretrained diffusion or flow-based policies using task-level feedback while preserving their sampling procedures. DDPO optimizes diffusion policies with policy gradients [32], Flow-GRPO introduces SDE-based formulations for flow optimization [13], and π RL extends flow-based RL to VLA models [33]. However, these methods typically rely on explicit sampling formulations that are not directly available for the high-order solvers used by recent world-action models. FPO addresses this limitation by deriving an advantage-weighted surrogate from

conditional flow-matching losses while treating the sampler as a black box [14]. Unlike WAM-RL, which relies on environment interaction and jointly updates the actor and world model [34], we perform offline post-training with frozen predictive models and optimize only the action head using FPO under Cosmos3’s native UniPC sampler.

III. PRELIMINARIES

A. Flow-matching generative policies

Flow matching trains continuous normalizing flows by regressing a time-dependent vector field along prescribed conditional probability paths between noise and data. Given a condition c , the learned field v_θ defines the ordinary differential equation

$$z_0 \sim p_0, \quad \frac{dz_\tau}{d\tau} = v_\theta(z_\tau, \tau | c), \quad \tau \in [0, 1], \quad (1)$$

where p_0 is the base distribution and z_τ the state at time τ along the trajectory integrated from z_0 , with $\tau = 0$ and 1 corresponding to noise and the generated endpoint.

B. Flow Policy Optimization

Flow Policy Optimization (FPO) constructs a surrogate policy ratio from conditional flow-matching (CFM) losses, avoiding the need to evaluate the exact likelihood of a flow-based policy. For a generated sample z under condition c , let $\hat{L}_{\text{CFM},\theta}(z | c)$ denote an empirical estimate of its CFM loss under parameters θ . FPO defines the surrogate ratio as

$$\hat{\rho}_\theta^{\text{FPO}}(z | c) = \exp \left[\hat{L}_{\text{CFM},\theta_{\text{old}}}(z | c) - \hat{L}_{\text{CFM},\theta}(z | c) \right], \quad (2)$$

where θ_{old} generated z and θ is the current policy being optimized. Both losses use the same sample, condition, and time–noise pairs. This ratio is a surrogate for policy change, not the exact likelihood ratio $p_\theta(z | c)/p_{\theta_{\text{old}}}(z | c)$.

IV. METHOD

Fig. 2 overviews our world-model-guided reinforcement-learning framework. The world-action model generates a goal-conditioned visual prediction and an action chunk; a frozen action-conditioned world model predicts the visual consequences of that action chunk and supplies the next observation. The discrepancy between the two predictions, together with alignment to a terminal visual goal, forms the feedback signal used to optimize the action head, avoiding additional real-robot interaction or training of task-specific reward models.

A. Problem Formulation

At rollout step k , the world-action model samples a latent endpoint z_k conditioned on the current observation o_k and instruction g . Denote its distribution as Π_θ^z and the decoding mapping as D , which outputs the generated visual prediction V_k^C and action chunk $a_k \in \mathbb{R}^{H_a \times d_a}$:

$$z_k \sim \Pi_\theta^z(\cdot | o_k, g), (V_k^C, a_k) = D(z_k), \quad (3)$$

The action chunk is converted into the control representation required by the pretrained frozen action-conditioned world

model W_{ϕ^0} , which predicts the action-conditioned visual consequence.

$$V_k^W = W_{\phi^0}(o_k, T_a(a_k)), \quad (4)$$

where T_a denotes the action conversion and temporal resampling interface. The two visual predictions are complementary: V_k^C is conditioned on the task goal, V_k^W on the proposed actions.

We partition the world-action model parameters into the frozen video-generation parameters θ_v^0 and the trainable action-related parameters θ_a , with $\theta = (\theta_v^0, \theta_a)$. Policy improvement does not require real-robot transitions or task-success labels; the framework instead constructs a model-mediated trajectory

$$X = \{(o_k, a_k, V_k^C, V_k^W)\}_{k=0}^{N-1}$$

and optimizes

$$\theta_a^* = \arg \max_{\theta_a} \mathbb{E}_{X \sim p_{\theta_a}(X | o_0, g; \theta_v^0, \phi^0)} \left[\sum_{k=0}^{N-1} \gamma^k r_k \right], \quad (5)$$

with θ_v^0 and ϕ^0 fixed throughout training. The reward r_k and transition to o_{k+1} are defined in (10) and (6).

B. Method Overview

The core idea is to use predicted action consequences as feedback, encouraging actions whose predicted outcomes align with the model’s goal-conditioned visual predictions. At each chunk, the WAM generates a goal-conditioned visual prediction and action sequence, while a frozen action-conditioned world model predicts their consequences and provides the next observation. The feedback combines trajectory-level consistency with terminal visual-goal alignment. We optimize only the action head using an FPO surrogate with group-relative advantages, while keeping the visual path and world model frozen. Algorithm 1 summarizes the training procedure.

C. Closed-Loop Cross-Model Rollout

We instantiate the world action model Π_θ with a native Cosmos3 policy [5] and the action-conditioned world model W_{ϕ^0} with a frozen Ctrl-World model [25].

For each policy step, Cosmos3 jointly generates the goal-conditioned visual prediction V_k^C and action chunk a_k in (3). Before passing to Ctrl-World, T_a restores the physical scale of the actions, converts the policy representation into the world-model control interface, and resamples the commands to the world-model rate. Ctrl-World then generates the consequence video V_k^W from the current observation and the converted actions.

The rollout is closed by using the predicted consequence to construct the observation for the next policy step:

$$o_{k+1} = \text{Ext}_{\text{obs}}(V_k^W), \quad (6)$$

where Ext_{obs} extracts the latest multi-view observation after the action chunk. The world model retains image and action histories within each rollout and updates them after each chunk; we omit these history inputs for brevity. Importantly,

V_k^C serves only as the target for cross-model feedback and is never fed back as the next observation. Repeating (3)–(6) for N chunks yields a closed-loop trajectory in which subsequent actions are conditioned on the predicted consequences of earlier actions.

For group-relative optimization, we sample B groups, each containing G trajectories that share the same reset observation and instruction:

$$\begin{aligned} o_{b,i,0} &= o_{b,0}, & g_{b,i} &= g_b, \\ b &= 1, \dots, B, & i &= 1, \dots, G, \end{aligned} \quad (7)$$

Within each group, independent draws from p_0 yield different latent endpoints and hence different imagined futures and action proposals under the same initial task condition.

Algorithm 1 Closed-loop post-training

Require: policy Π_{θ_a} , frozen world model W , reset set D_{reset} , goal image I_g^* , group size G , horizon N , discount γ

- 1: $\theta_{a,\text{old}} \leftarrow \theta_a$; sample $(o_0, g) \sim D_{\text{reset}}$
- 2: **for** $i = 1, \dots, G$ **do**
- 3: $o_{i,0} \leftarrow o_0$
- 4: **for** $k = 0, \dots, N-1$ **do**
- 5: $(V_{i,k}^C, a_{i,k}, z_{i,k}) \leftarrow \text{NativeUniPC}(\Pi_{\theta_{a,\text{old}}}, o_{i,k}, g)$
- 6: $V_{i,k}^W \leftarrow W(o_{i,k}, T_a(a_{i,k}))$
- 7: Compute $r_{i,k}$
- 8: Store $z_{i,k}, c_{i,k} = (o_{i,k}, g)$, and fixed probes $\Omega_{i,k}$
- 9: $o_{i,k+1} \leftarrow \text{Ext}_{\text{obs}}(V_{i,k}^W)$
- 10: **end for**
- 11: **end for**
- 12: Compute $R_{i,k}$ and $\hat{A}_{i,k}$
- 13: **for** each minibatch of stored chunks **do**
- 14: **for** each chunk (i, k) in the minibatch **do**
- 15: Compute $\hat{L}_{\text{old}}^a$ and $\hat{L}^a$
- 16: $\hat{\rho}_{i,k} \leftarrow \exp(\hat{L}_{\text{old}}^a - \hat{L}^a)$
- 17: **end for**
- 18: Update θ_a once using the minibatch average
- 19: **end for**

D. Cross-Model Feedback and Temporal Credit

We construct feedback from trajectory consistency and terminal-goal alignment. For trajectory i in group b at chunk k , the trajectory feedback measures the discrepancy between the WAM-generated visual prediction and the action-conditioned prediction. We select frames at H_v common timestamps from both videos and compare corresponding views at matching resolution. For vectorized frames F, F' , we use pixel MSE, $D_{\text{traj}}(F, F') = D_{\text{goal}}(F, F') = \|F - F'\|_2^2 / P$, where P counts all compared pixel values across channels and views, and pixel values are on the $[0, 255]$ scale. The trajectory reward is

$$r_{b,i,k}^{\text{traj}} = -\frac{1}{H_v} \sum_{t=1}^{H_v} D_{\text{traj}}(V_{b,i,k,t}^C, V_{b,i,k,t}^W). \quad (8)$$

To encourage task completion, we additionally compare V_{final}^W , the final frame predicted by the world model, with a

predefined visual goal reference I_g^* for each task g , preprocessed identically to the predicted frames. Omitting group and trajectory indices in (9)–(10), the goal reward is

$$r_k^{\text{goal}} = -D_{\text{goal}}(V_{\text{final}}^W, I_g^*), \quad (9)$$

The reward used for policy optimization is

$$r_k = \lambda_{\text{traj}} r_k^{\text{traj}} + \lambda_{\text{goal}} r_k^{\text{goal}}, \quad (10)$$

where the nonnegative weights λ_{traj} and λ_{goal} balance the two terms, and the goal term equals (9) at the final chunk and is zero otherwise. Higher feedback scores indicate smaller pixel discrepancies: the trajectory term encourages agreement with the WAM’s imagined motion, the goal term encourages progress toward the desired outcome.

A trajectory contains N consecutive action chunks. To assign an earlier chunk credit for the consequences predicted at later rollout steps, we compute the discounted suffix return

$$R_{b,i,k} = \sum_{j=k}^{N-1} \gamma^{j-k} r_{b,i,j}, \quad (11)$$

Returns are normalized separately at each chunk position across the G trajectories sharing the same reset state:

$$\hat{A}_{b,i,k} = \frac{R_{b,i,k} - \mu_{b,k}}{s_{b,k} + \delta}, \quad (12)$$

where $\mu_{b,k}$ and $s_{b,k}$ are the mean and population standard deviation of the G returns within group b at chunk k , and $\delta > 0$ ensures numerical stability.

E. Action-Head Policy Optimization

The group-relative advantages defined above provide the feedback for updating the WAM’s action head. We use them in a clipped FPO objective whose surrogate ratio is computed from action-head CFM losses, connecting the cross-model feedback to policy optimization without requiring gradients through the reward computation or the action-conditioned world model.

For member i of group b at chunk k , the WAM generates a joint latent endpoint $z_{b,i,k} = [z_{b,i,k}^v; z_{b,i,k}^a]$ using its native UniPC sampler, which we evaluate with the FPO surrogate introduced in the preliminaries [14]. This retains the native rollout procedure without requiring a likelihood over the numerical solver trajectory.

For each stored endpoint, we sample Q time–noise pairs $\Omega_{b,i,k} = \{(\tau_q, \varepsilon_q)\}_{q=1}^Q$, where $\tau_q \sim \text{U}(0, 1)$ and $\varepsilon_q \sim \text{N}(0, I)$ are independent and shared between the behavior and current policies. Following the noise-to-data convention, the perturbed input is

$$z_{b,i,k}^{(q)} = z_{b,i,k} + (1 - \tau_q) M_{b,i,k} \odot (\varepsilon_q - z_{b,i,k}), \quad (13)$$

The mask $M_{b,i,k}$ selects future visual latents and valid action coordinates, leaving conditioning and padding unchanged.

We compute the CFM loss only on valid action coordinates:

$$\hat{L}_{\theta_a, b, i, k}^a = \frac{1}{Q} \sum_{q=1}^Q \left\| P_a \left[v_{\theta_a^0, \theta_a} \left(z_{b,i,k}^{(q)}, \tau_q \mid c_{b,i,k} \right) - u_{b,i,k}^{(q)} \right] \right\|_2^2, \quad (14)$$

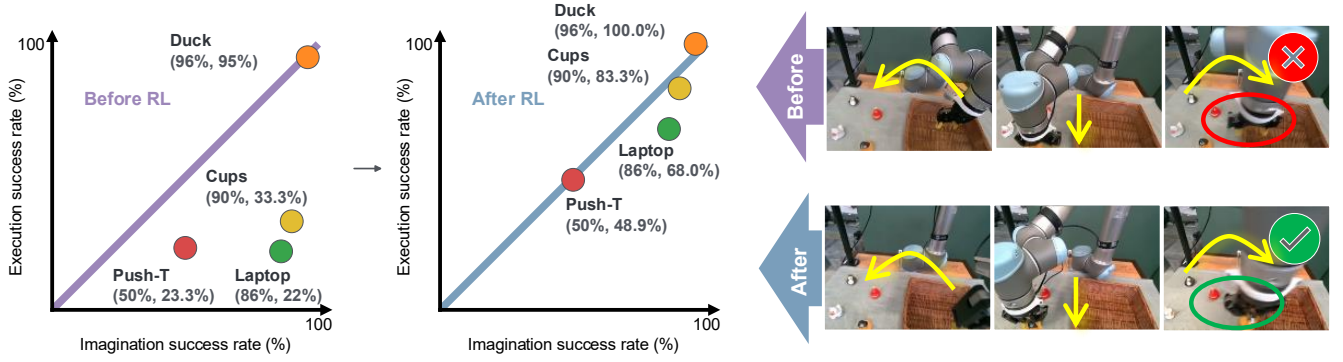


Fig. 3. **Imagination and execution success rates before (left) and after (right) RL.** Each point is one task; the diagonal marks perfect agreement, so distance below it is the imagination–consequence gap. The gap between imagination and execution success rates narrows after post-training. Far right: a duck grasp that fails before RL and succeeds after.

where $u_{b,i,k}^{(q)} = M_{b,i,k} \odot (z_{b,i,k} - \epsilon_q)$ is the target velocity of (13), P_a selects valid action coordinates, and $c_{b,i,k} = (o_{b,i,k}, g_b)$. This loss defines the surrogate ratio for updating the action head:

$$\hat{\rho}_{b,i,k}^{\text{FPO}}(\theta_a) = \exp(\hat{L}_{\theta_{a,\text{old}},b,i,k}^a - \hat{L}_{\theta_a,b,i,k}^a), \quad (15)$$

where both losses use the same endpoint, condition, and time–noise pairs.

We combine this ratio with the group-relative advantage $\hat{A}_{b,i,k}$ and maximize the critic-free clipped objective:

$$J_{\text{FPO}}(\theta_a) = \mathbb{E} \left[\min \left(\rho \hat{A}, \text{clip}(\rho, 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A} \right) \right], \quad (16)$$

where ρ and $\hat{A}$ denote the ratio in (15) and the advantage in (12), the expectation averages uniformly over the *BGN* collected chunks, and $\epsilon_{\text{clip}} \in (0, 1)$ is the clipping threshold. The sampled trajectories, probe pairs, advantages, and behavior-policy losses remain fixed during optimization. Only the action-head parameters θ_a are updated; the WAM’s visual-generation parameters and the action-conditioned world model stay frozen, preventing direct updates to the predictive models during optimization.

V. EXPERIMENTS

We evaluate the proposed framework through four questions. **Q1:** How consistent are the visual predictions of a world–action model with the action-conditioned consequences predicted from its generated actions? **Q2:** Can policy optimization guided by this model-derived feedback improve real-world robot performance? **Q3:** How do trajectory-level and terminal-goal feedback components affect policy optimization? **Q4:** How do model-derived feedback signals relate to predicted task success? To address these questions, we focus on four aspects: model consistency, real-robot performance, feedback mechanisms, and reward effectiveness.

A. Experimental Setup

1) *Tasks:* As shown in Fig. 4, we evaluate four real-world manipulation tasks on a UR5 robot:

- *Laptop closing:* The robot rotates the open lid until the laptop is fully shut without displacing the base (40 demonstrations).

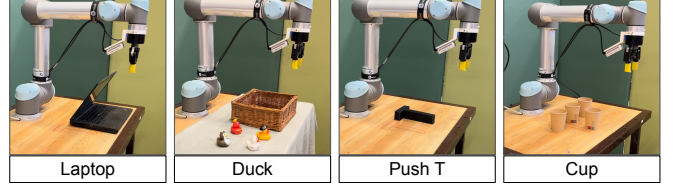


Fig. 4. **Experimental tasks:** closing a laptop, placing a specified colored duck into a basket (four colors, selected from the instruction), pushing a T-shaped block to a target pose, and nesting four cups into one stack. We collect 40, 200, 150, and 100 demonstrations for the respective tasks; Cup has the longest horizon.

- *Duck placing:* The robot grasps the colored duck specified by the instruction from four candidates on the table and places it into a basket. All four colors appear during training (200 demonstrations, 50 per color).
- *Push-T:* The robot translates and reorients a T-shaped block to match a target pose (150 demonstrations).
- *Cup nesting:* The robot stacks four cups into a single column, requiring four successive grasp-and-place motions without disturbing the partial stack (100 demonstrations).

Laptop closing and cup nesting require accurate contact establishment and maintenance during execution, while Push-T and duck placing require visual inference of object pose and identity.

2) *Compared methods:* We compare the full framework against its SFT initialization and three external systems: FastWAM [20], VLA-JEPA [24] and $\pi_{0.5}$ [35]. Each retains its official model, training procedure, observation and action interface, and default execution horizon, while being adapted and evaluated using the same demonstration data and task protocol.

3) *Training and checkpoint selection:* All systems are adapted from the same task-specific UR5 demonstration split, divided into training and testing sets at a 4:1 ratio. For each method, we select the checkpoint with the lowest validation loss during adaptation.

4) *Implementation details:* We combine FPO with group-relative advantages. Each rollout batch contains $B = 8$ groups of $G = 16$ trajectories that share the same initial observation and instruction and differ only through independent pol-

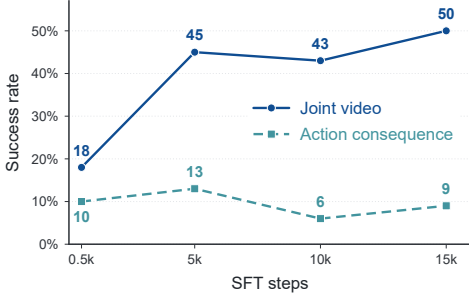


Fig. 5. **Model-space success during SFT.** Both streams are scored by a frozen classifier on the terminal frame. The imagined video increasingly shows task completion (18 $\rightarrow$ 50%) while the world model’s rollout of the same actions does not (6–13%), leaving a 41-point gap at 15k steps.

icy sampling noise. Each trajectory contains $N = 5$ action chunks, and discounted suffix returns are normalized within each group at each chunk position with $\gamma = 0.95$.

B. Model-Space Video–Consequence Gap During SFT

We first test whether task-completion predictions from the WAM visual stream are consistent with consequences predicted from generated actions. At four SFT checkpoints (500, 5k, 10k, and 15k steps), we sample 100 paired rollouts from 20 initializations. Within each pair, the video stream recursively conditions on its predictions while the frozen world model predicts the consequences of the same action chunks from the same initial observation, and a frozen instruction-conditioned classifier scores the terminal predictions.

Fig. 5 shows the resulting model-space success scores. As SFT proceeds, the WAM visual predictions increasingly depict task completion, rising from 18% to 50%, while the corresponding action-conditioned predictions remain at 6–13% throughout. The paired gap widens to 41 percentage points at 15k steps. These results indicate an asymmetry between visual prediction capability and action generation capability: the model can produce task-completion visual predictions before generating action sequences whose predicted consequences achieve similar outcomes. Because both scores are obtained using the same classifier and initial states, this gap reflects inconsistency between two model-generated predictions rather than differences in evaluation protocol, and it motivates the feedback defined in Eq. (10).

C. Overall Real-Robot Performance

Table I reports real-robot success rates, averaged over four tasks with equal weight. After RL, our model achieves the highest average success rate among evaluated methods at 75.1%, improving over its SFT initialization (43.4%) by 31.7 percentage points and on the strongest baseline, $\pi_{0.5}$, by 13.7. The results show policy optimization with world-model feedback provides gains beyond supervised imitation alone.

The improvement is consistent across all four tasks: 22.0% to 68.0% on laptop, 95.0% to 100.0% on duck, 23.3% to 48.9% on Push-T, and 33.3% to 83.3% on cup. Compared with $\pi_{0.5}$, our method achieves higher success rates on all evaluated tasks, with gains of 11.3, 15.0, 15.6, and 12.7

TABLE I
REAL-ROBOT TASK SUCCESS RATES (%).

Method	Laptop	Duck	Push-T	Cup	Avg.
Ours (SFT)	22.0%	95.0%	23.3%	33.3%	43.4%
Fast-WAM	16.0%	0.0%	0.0%	0.0%	4.0%
VLA-JEPA	15.0%	20.0%	0.0%	0.0%	8.8%
$\pi_{0.5}$	56.7%	85.0%	33.3%	70.6%	61.4%
Ours (RL)	68.0%	100.0%	48.9%	83.3%	75.1%

percentage points, respectively. Under the same adaptation protocol, VLA-JEPA and Fast-WAM achieve lower success rates than our SFT initialization. Both were developed with substantially larger adaptation budgets than the 40–200 demonstrations per task available here, so we read these numbers as evidence that the low-data regime is difficult rather than as an inherent limitation of either method.

Table II shows that execution efficiency also improves after RL. Mean completion time and mean control step count decrease on all four tasks: by 10.2% and 9.9% on laptop, 10.7% and 14.7% on duck, 6.0% and 3.8% on Push-T, and 16.8% and 25.1% on the long-horizon cup task. The medians follow the same trend except on duck, where they are essentially unchanged, indicating that the mean reduction there comes from fewer long, hesitant trials rather than a uniformly faster policy. These statistics are computed over each policy’s successful trials, so the two rows cover different trial sets; the consistent direction across tasks nonetheless suggests that improving consistency between generated actions and predicted visual outcomes reduces corrective motion rather than trading accuracy for speed.

TABLE II
REAL-ROBOT TASK EXECUTION EFFICIENCY.

Task	Method	n	Time (s) $\downarrow$		Control steps $\downarrow$	
			Mean	Median	Mean	Median
Laptop	SFT	11	46.1	40.8	123.6	113
	RL	34	41.4	39.7	111.4	107
Duck	SFT	57	63.5	54.1	200.0	160
	RL	60	56.7	54.2	170.7	160
Push-T	SFT	21	131.6	124.4	289.3	279
	RL	44	123.7	121.9	278.3	272
Cup	SFT	10	206.0	196.7	667.6	632
	RL	25	171.3	173.4	500.0	496

Statistics use each policy’s successful trials; n is the number of successes. Time includes inference waiting. Control steps count executed actions.

D. Ablation Study of Reward

We evaluate the respective contributions of trajectory-level and terminal-goal feedback through a feedback-component ablation on the *close-laptop* task, keeping the SFT initialization, Ctrl-World model, UniPC sampler, FPO objective, and training budget constant and varying only the reward composition. We compare three settings: *Trajectory reward*, which uses only the trajectory-consistency term; *Terminal reward*, which uses only the terminal-goal term; and *Combined reward*, which integrates both.

As shown in the left three panels of Fig. 6, all three settings improve steadily over FPO updates, each rising under its

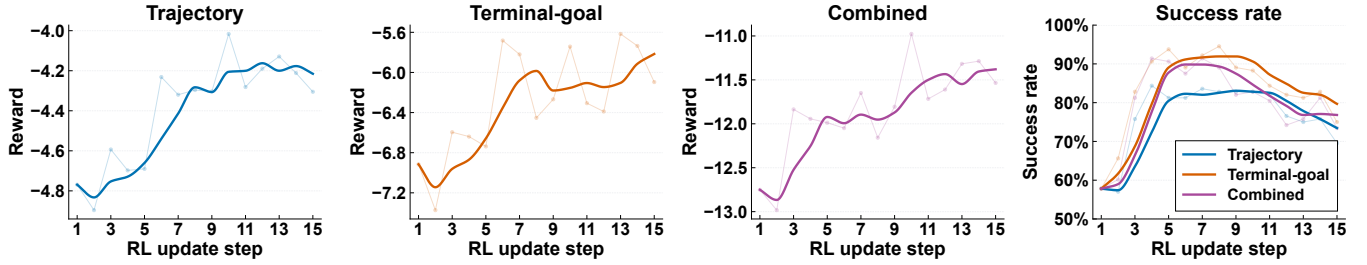


Fig. 6. **Reward ablation on the close-laptop task, varying only the reward composition.** Left three: Trajectory, Terminal-goal, and Combined rewards over FPO updates, each rising under its own objective. Right: classifier-estimated model-space success, which improves early then diverges — model-space success does not directly track real-robot performance.

own objective. Both feedback types therefore provide useful optimization signals for FPO individually. On the right of Fig. 6, all three settings improve sharply in the early updates, peak around updates 9–11, and then decline. Because this metric comes from a classifier applied to Ctrl-World’s predicted terminal states, it measures completion only within the model space, and the late decline may indicate a mismatch between the optimized policy and the prediction distribution of the frozen world model rather than genuine degradation. We accordingly use model-space success only to compare optimization signals during training.

These results suggest that the two feedback components play complementary roles: the trajectory feedback constrains consistency during intermediate predicted evolution, whereas the terminal feedback focuses on the final predicted goal state. Combining feedback from both the process and the endpoint yields a more comprehensive optimization signal.

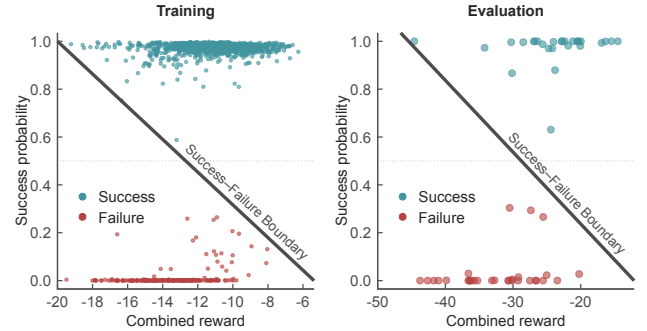


Fig. 8. **Combined reward versus predicted success probability on the training (left) and evaluation (right) sets;** each point is one rollout. Successes concentrate at high probability and higher reward, failures at the opposite corner, with similar separation in both splits. Diagonals are visual guides only; both axes come from Ctrl-World predictions, so the correlation is within model space.

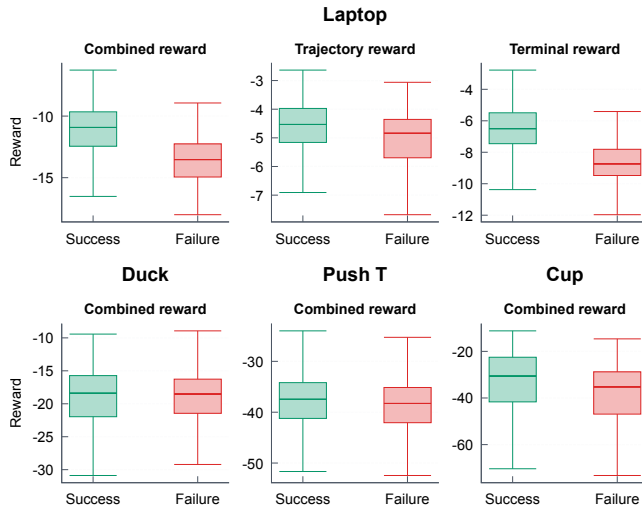


Fig. 7. **Reward distributions for successful (green) and failed (red) rollouts.** Laptop is shown for all three reward settings; other tasks use the combined reward. Successes score higher throughout, but the margin is clear on Laptop and Cup and largely overlapping on Duck and Push-T.

E. Correlation Between Reward and Predicted Success

We analyze the relationship between the feedback signal and task-prediction success. For each RL rollout, we pair the feedback score with the success classification derived from the final-state image predicted by Ctrl-World, so this analysis reflects the association within the model space. As

shown in Fig. 7, this relationship varies across tasks. On laptop and cup, reward distributions for successful rollouts sit clearly above those for failed ones, whereas on duck and Push-T the two overlap substantially. The combined feedback therefore carries directional information consistent with predicted success, though the association strength is task-dependent. We further examine the relationship at the level of individual rollouts. As shown in Fig. 8, successful rollouts concentrate at high predicted success probability and at the upper end of the reward range, while failed rollouts fall at the opposite corner, with similar separation in the training and evaluation splits. The combined reward thus orders rollouts consistently with predicted task success, supporting its use as a feedback signal. Because both quantities derive from the same Ctrl-World predictions, this correlation holds within the model space.

Both components show consistent trends: trajectory feedback captures consistency between predicted action evolution and target progression, while terminal feedback emphasizes final-state completion. Combining them supervises both intermediate process and final outcome. Higher feedback scores correspond to more successful model predictions across reward distributions and individual rollouts; although task-dependent, the feedback remains informative for guiding the policy toward improved predicted outcomes.

VI. CONCLUSIONS

We propose a policy optimization method that converts the discrepancy between a world-action model’s visual pre-

dictions and action-conditioned consequences into an optimization signal for its actions. A frozen pretrained action-conditioned world model predicts the action-conditioned visual consequences of generated actions, and the discrepancy between the WAM visual prediction and the predicted consequence, together with alignment to a terminal visual goal, forms a dense rollout-level feedback signal for policy optimization without directly updating the predictive models. Adapting Flow Policy Optimization to the action head allows this feedback signal to update the policy under the pretrained model's native UniPC sampler, without online robot interaction or additional task-specific reward-model training. On four real-world UR5 tasks, post-training on a 41-point visual prediction-consequence gap raises the average success rate from 43.4% to 75.1%, and ablations confirm the complementary roles of trajectory-level and terminal-goal feedback. Future work will explore world models that explicitly incorporate physical constraints to better assess the feasibility of visual plans and narrow the gap between visual planning and real-world execution, while extending the approach to longer-horizon manipulation.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choremanski, T. Ding, D. Driess, A. Dubey, C. Finn *et al.*, "Rt-2: Vision-language-action models transfer web knowledge to robotic control," *arXiv preprint arXiv:2307.15818*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi *et al.*, "Openvla: An open-source vision-language-action model," 2024.
- [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "*pi.0*: A vision-language-action flow model for general robot control," 2024.
- [4] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel, "Learning universal policies via text-guided video generation," vol. 36, 2023, pp. 9156–9172.
- [5] N. Agarwal, A. Ali, J. Allen, M. Antolini, A. Aubame, A. Azzolini, J. Bai, M. Bala, Y. Balaji, J. Bapst *et al.*, "Cosmos 3: Omnimodal world models for physical ai," 2026.
- [6] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu *et al.*, "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," 2024.
- [7] I. Xu, S. Jiang, J. Marangola, N. Dashora, R. Li, T. Liu, Z. He, Y. Zhi, A. Pentland, P. Agrawal *et al.*, "Learning more from less: Reinforcement learning from hindsight," 2026.
- [8] Y. Guo, J. Gao, Z. Wu, C. Shi, and J. Chen, "Reinforcement learning with demonstrations from mismatched task under sparse reward," in *Conference on Robot Learning*. PMLR, 2023, pp. 1146–1156.
- [9] Y. Wang, Z. Sun, J. Zhang, Z. Xian, E. Biyik, D. Held, and Z. Erickson, "Rl-vlm-f: Reinforcement learning from vision language foundation model feedback," 2024.
- [10] F. Luo, "Vision-language models for robot success detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 21, 2024, pp. 23 750–23 752.
- [11] R. Tian, Y. Wu, C. Xu, M. Tomizuka, J. Malik, and A. Bajcsy, "Maximizing alignment with minimal feedback: Efficiently learning rewards for visuomotor robot policy alignment," 2024.
- [12] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, Q. Zhang, Z. Yu, G. Fan *et al.*, " π rl: Online rl fine-tuning for flow-based vision-language-action models," 2025.
- [13] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang, "Flow-grpo: Training flow matching models via online rl," vol. 38, 2026, pp. 40 783–40 818.
- [14] D. McAllister, S. Ge, B. Yi, C. M. Kim, E. Weber, H. Choi, H. Feng, and A. Kanazawa, "Flow matching policy gradients," pp. 36 352–36 372, 2026.
- [15] S. L. Li, E. Kim, X. Bai, T. Zhao, T. Pang, M. Simchowitz, and V. Sitzmann, "Turning video models into generalist robot policies," *arXiv preprint arXiv:2605.27817*, 2026.
- [16] S. Zhou, Y. Du, J. Chen, Y. Li, D.-Y. Yeung, and C. Gan, "Robodreamer: Learning compositional world models for robot imagination," 2024.
- [17] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong, "Unleashing large-scale video generative pre-training for visual robot manipulation," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 10 641–10 662.
- [18] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn *et al.*, "Cosmos policy: Fine-tuning video models for visuomotor control and planning," 2026.
- [19] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang *et al.*, "World action models are zero-shot policies," *arXiv preprint arXiv:2602.15922*, 2026.
- [20] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, "Fast-wam: Do world action models need test-time future imagination?" 2026.
- [21] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus *et al.*, "V-jepa 2: Self-supervised video models enable understanding, prediction and planning," *arXiv preprint arXiv:2506.09985*, 2025.
- [22] J. Chen, K. Wang, K. Chen, S. Chen, F. Gao, W. Tang, Z. Li, W. Liu, Z. Yao, B. Li *et al.*, "Lawam: Latent world action models for efficient dynamics-aware robot policies," *arXiv preprint arXiv:2606.15768*, 2026.
- [23] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "Dino-wm: World models on pre-trained visual features enable zero-shot planning," 2024.
- [24] J. Sun, W. Zhang, Z. Qi, S. Ren, Z. Liu, H. Zhu, G. Sun, X. Jin, and Z. Chen, "Vla-jepa: Enhancing vision-language-action model with latent world model," 2026.
- [25] Y. Guo, L. Shi, J. Chen, and C. Finn, "Ctrl-world: A controllable generative world model for robot manipulation," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 6121–6138.
- [26] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, and M. Long, "ivideogpt: Interactive videogpts are scalable world models," vol. 37, 2024, pp. 68 082–68 119.
- [27] S. Yang, Y. Du, K. Ghasemipour, J. Tompson, L. Kaelbling, D. Schuurmans, and P. Abbeel, "Learning interactive real-world simulators," 2023.
- [28] F. Zhu, H. Wu, S. Guo, Y. Liu, C. Cheang, and T. Kong, "Irasim: A fine-grained world model for robot manipulation," in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2025, pp. 9834–9844.
- [29] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps *et al.*, "Genie: Generative interactive environments," in *Forty-first international conference on machine learning*, 2024.
- [30] S. Gao, S. Zhou, Y. Du, J. Zhang, and C. Gan, "Adaworld: Learning adaptable world models with latent actions," 2025.
- [31] W. Menapace, S. Lathuiliere, A. Siarohin, C. Theobalt, S. Tulyakov, V. Golyanik, and E. Ricci, "Playable environments: Video manipulation in space and time," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 3574–3583.
- [32] K. Black, M. Janner, Y. Du, I. Kostrikov, and S. Levine, "Training diffusion models with reinforcement learning," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 4965–4987.
- [33] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin, H. Zang, X. Li, Q. Zhang, Z. Yu *et al.*, " π_{RL} : Online rl fine-tuning for flow-based vision-language-action models," *arXiv preprint arXiv:2510.25889*, 2025.
- [34] Z. Qian, X. Chi, Y. Qi, H. Li, Z. Y. Chen, and S. Zhang, "Wam-rl: World-action model reinforcement learning with reconstruction rewards and online video sft," 2026.
- [35] P. Intelligence, K. Black, N. Brown, J. Darpanian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai *et al.*, " $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization," *arXiv preprint arXiv:2504.16054*, 2025.